

樹搜尋原型可行性報告(Phase C-3 + D-7:v1+v2 全 8 場)

產生方式:數字逐字取自 `tree_search/harness.py` (v1)、`tree_search/harness_v2.py` (v2)、八份 `competitions/playground-series-{s3e9,s3e14,s3e5,s3e3,s3e7,s3e1,s3e19,s3e11}/experiments_tree.json`,以及對應 `STATUS.md` 的樹搜尋附錄;抽取腳本 `docs/scripts/build_tree_facts.py` → `docs/tree_facts.json`。計畫書脈絡:ERA(Aygün et al. 2026, Nature)——用候選樹搜尋取代線性單路徑迭代,核心機制為節點評分、選擇規則、plateau 偵測與回溯、以及「想法注入」持續擴充候選集。第 1–5 節(Phase C-3)是計畫書 Stage 4(週 6–7)的先遣驗證,在 s3e9/s3e14/s3e5 三場上初步回答「樹搜尋值不值得投入」(1 勝 1 平 1 負,結論:值得,但需先修第 5 節列出的缺口)。第 6–9 節(Phase D-7)是該驗證通過後,依第 6 節(原第 5、6 節)建議實作 `harness_v2` 四項升級,並在 s3e3/s3e7/s3e1/s3e19/s3e11 五場(涵蓋 AUC/RMSE/SMAPE/RMSLE 四個指標家族、KFold 與 TimeSeriesSplit 兩種 CV 方案、1,677–360k 列三個資料規模)上做全面掃描的結果——**v1+v2 合計 8 場**,問題從「值不值得做」升級為「在什麼條件下、用多少評估數就能贏,以及贏的來源是先驗還是在地洞見」。

1. 目的與設計

設計目標

`tree_search/harness.py` 是一個與競賽無關的通用引擎;每場競賽只需提供一個 `evaluate(config) -> score` 函式(`tree_search/eval_s3e9.py` / `eval_s3e14.py` / `eval_s3e5.py`)。設計選擇如下:

- **node = 一個完整、已評分(或已失敗)的解**: `{id, parent_id, mutation, config, score, status, wall_s}`。不是「部分想法」或「一個超參數變動」的抽象節點,而是一個可獨立重現、有 CV 分數的完整 pipeline 設定——這是刻意的簡化,換取「隨時可比較任兩個節點」的簡單性。
- **選擇規則(selection rule)**:任一時刻恰有一條「活躍 lineage」(root 的某個第一代子節點所代表的候選子樹)。`select_next_parent()` 永遠在活躍 lineage 內,挑選「分數最好、且尚未達到 `MAX_CHILDREN_PER_NODE` (=3)個子節點」的節點來擴展。
- **plateau/backtrack**:每當活躍 lineage 新增一個子節點,若其分數未能刷新「新增前的全域最佳分數」,該 lineage 的未改進計數 +1;連續達到 `PLATEAU_STREAK` (=3)次,該 lineage 標記為「已 plateau」並從候選中剔除, `select_next_parent()` 改選次佳、尚未 plateau 的 lineage(即回

溯)。兩個額外規則防止搜尋卡死:(1) 一條 lineage 若耗盡擴展預算(每個節點都已有 3 個子節點)也視為 plateau;(2) 若所有 lineage 同時 plateau,清空 plateau 旗標一次(reopen),讓搜尋能繼續花完節點預算。

- **solo + blend 雙節點空間(Phase C-2b 起):** `eval_s3e14.py` / `eval_s3e5.py` 支援兩種節點型別—— `solo` (訓練一個模型,OOF/test 預測快取到磁碟)與 `blend` (對已快取的成員 OOF 做權重搜尋,不重訓)。這是修正 s3e9 教訓的關鍵設計:單模型節點空間永遠碰不到集成解法能到達的分數。
- **OOF 快取:** `harness.next_id()` (公開包裝)讓 `solo` 節點在 `add_node()` 真正指派 id 之前就能預先知道自己的 id,把 OOF/test 矩陣存成 `tree_search/cache_<comp>/solo_<node_id>.npz`,後續 `blend` 節點直接讀取、不必重新訓練——這是 `blend` 節點能做到「近乎免費評分」的唯一原因。

與 ERA 的對應與簡化

ERA(Aygün et al. 2026)理念	本原型的對應	簡化了什麼
候選樹(candidate tree)取代單路徑迭代	<code>harness.py</code> 的 <code>node/lineage/select_next_parent</code> 機制,完整實作	無
Plateau 偵測與回溯	<code>PLATEAU_STREAK (=3)+ lineage 排除 + reopen-once</code> 保底	固定閾值,未依指標特性(離散 vs 連續)動態調整——見第 4、6 節
想法注入(idea injection,持續產生新候選方向)	未實作: 每條 lineage 的變異佇列(mutation queue)是驅動腳本(<code>run_s3e9.py</code> 等)裡手寫的固定清單,佇列用盡後退化為「換 random seed」的填充策略	ERA 的第二支柱(見第 5 節已知缺口)
多機並行搜尋候選子樹	單機序列: <code>harness.py</code> 的 docstring 明載「no concurrent add_node calls」, <code>add_node / save</code> 假設嚴格單一寫入者	無平行化,樹的深度/廣度受限於單一序列執行時間
變異提案者 (mutation proposer) 為系統的一部分、可自我演化	固定為 agent(Claude)在撰寫驅動腳本當下手動設計的規則式佇列 ,執行期間不再產生新想法,只依評分排序既有佇列	無運行時想法生成,退化為「窮舉一份預先寫好的候選清單,由樹搜尋機制決定探索順序」

簡言之:本原型完整驗證了 ERA 的**搜尋機制**(選擇/plateau/回溯/斷點續傳),但尚未觸及 ERA 真正被引用的差異化能力——**想法注入**。這是解讀第 2–4 節結果時的關鍵前提:任何「樹搜尋沒有贏」的案例,都可能是「候選集本身不夠豐富」造成,而非搜尋機制本身的缺陷。上表最後兩列(想法注入、變異提案者固定)正是第 6 節 `harness_v2` 四項升級中第 1、4 項的直接動機——見第 6 節。

2. v1:三場結果總表(Phase C-2a/b/c)

comp	metric	線性最佳	樹最佳	勝/平/負	節點數	超車節點數 (evaluation-to-beat)	回溯次數	牆鐘時間
s3e9	RMSE(min)	12.07003	12.07459	負	20	從未超車(僅重新發現既有單模最佳)	4	125.4s
s3e14	MAE(min)	340.59891	340.52635	勝	20	9(node #8, 340.59485 已超車;最終於 node #11 拉開 0.07256)	2	593.4s
s3e5	QWK(max)	0.56769	0.56766	平	40	從未超車,差距僅 0.00003	11(含 1 次 reopen)	696.9s

3. 逐場分析

s3e9(單模型空間困於噪音上限)

Root 為 s3e9 既有的正則化單一 LGB(RMSE 12.11061,5.2s);5 條第一代 lineage(CAT/XGB/FEAT/ROBUST/REG)全部是**單模型**變異。20 個節點全部評估成功(0 失敗),每節點成本 2.0–17.0s,總牆鐘 125.4s——比 20–60s/節點的設計目標更便宜。全樹最佳分數是 CatBoost 種子節點本身(12.07459),CAT 自己的 3 個子節點(加正則、縮容量、加 bagging_temperature)全部打平或輸給父節點;4 次回溯(CAT→REG→FEAT→XGB 依序 plateau)都沒能找到更好的方向。

教訓:此分數在單模型空間裡已是局部最優——線性迭代真正的最佳分數(12.07003)來自 7-way seed-bagged blend,樹搜尋因為節點空間限定在單模型,結構上就到不了那裡(樹最佳 12.07459 比線性最佳差 0.00456)。這不是搜尋機制失敗,是候選空間設計失敗——直接促成 s3e14 加入 blend 節點型別的設計決策。

s3e14(勝:blend 節點 <1s、fold-exact 驗證)

Root 沿用既有調校 LGB(342.02154,52.1s)。7 條第一代 lineage:6 條 solo(CAT/XGB/LGBTUNED/SEEDBAG/FEAT/REG,用來填充 OOF 快取池)+ 1 條 BLEND(對快取成員做 Dirichlet 或 grid-simplex 權重搜尋)。20 個節點、0 失敗,12 個 solo(19.8–81.4s)+ 8 個 blend(0.5–0.8s)。

BLEND 種子節點(3-way:root+XGB+CAT)一評分就是全樹最佳,harness 隨即持續拉著這條 lineage 擴展:加入 LGBTUNED 後在第 9 個評估(node #8, 340.59485)就**超車**了線性迭代耗費 7 次實驗(每次都是完整多模型 pipeline)才找到的 340.59891;之後加入 SEEDBAG(node #9, 340.54904)、加入 FEAT(node #11, **340.52635**,全樹最終最佳)。node #10(在同一組 5 個成員上把權重搜尋法從 dirichlet 換成 grid_simplex)重現了線性迭代 exp #7 的**逐位精確**結果(raw 340.65207 / snap 340.59891),確認兩套流程在同一組 5-fold 上完全可比——這是本原型唯一一次做到「fold-exact」交叉驗證。2 次回溯都發生在最佳分數已找到之後(node #14 BLEND lineage plateau、node #17 LGBTUNED lineage plateau),說明 plateau 規則正確地把剩餘預算轉去探索新的 solo 多樣性來源,而非在已飽和的 blend lineage 上空轉。

贏的原因:winning node(#11)的第 6 個成員——把 21 特徵集再砍 3 個溫度欄位的 LGB(solo 本身 342.02283,單模並無進步)——線性迭代從未試過這個成員組合;它的價值純粹是 blend 多樣性,只有節點空間能展開到 ensemble 才碰得到。

s3e5(平:成熟 blend 空間只被重新發現;離散指標階梯面)

Root 為 Optuna 直接以 post-rounder QWK 為目標調校出的 LGB(0.56244,1.4s)——線性迭代自己已經用最高槓桿的配方把單模天花板頂到位。8 條第一代 lineage(ROOTQ + 6 solo:CAT/XGB/LGBNUDGE/FEAT/CATORIG/LGBORIG + BLEND)。40 個節點、0 失敗:30 solo(0.6–27.1s)+ 10 blend(45.0–46.5s)。樹最佳是 node #11(4-way blend:root + tuned-CAT + XGB + FEAT-17特徵版,權重 0.129/0.645/0.018/0.208,cutpoints [3.590, 4.609, 5.614, 6.160, 7.597]),QWK **0.56766**——本質上以 4 個成員重新發現了線性迭代 6-way champion 的同一個最適區(該 champion 有效權重其實也只集中在 2 個成員上),與線性最佳 0.56769 只差 **0.00003**,量級小到單一樣本的切點歸屬就能翻轉。

離散指標帶來的三個新現象(此為本次泛化測試的核心產出):

1. **分數面呈階梯狀,平手極常見:**40 節點中出現 4 組 5 位小數完全相同的分數(0.56725×3 、 0.56664×2 、 0.56466×2 、 0.56065×2)。連續指標(s3e9 的 RMSE、s3e14 的 MAE)幾乎不會出現這種平手。
2. **PLATEAU_STREAK=3 對離散指標實質上更嚴格:**8 條 lineage 全數在本輪 plateau(這是三場裡最徹底的一次),必須觸發 reopen-once 規則才能把預算花完;reopen 後 BLEND lineage 吃進新出爐的 solo 池成員,得 0.56725,仍未超越 node #11。
3. **「blend 便宜」的假設不成立:**每個候選權重都要做一次 Nelder-Mead 擬合 OptimizedRounder,單一 blend 節點成本升到 ~45–46s,與 solo 節點(0.6–27.1s)同量級甚至更貴——s3e14 建立的「blend 節點 <1s、近乎免費」經驗無法直接遷移到離散指標場景。

一個順帶發現的實作缺陷:同一 parent 的 blend fallback 因 parent config 不變而重複產出同一組成員(node #37/#38/#39,同分 0.56725),在同分頻繁的離散面下浪費了 plateau 額度——已記錄於第 5 節「已知缺口」。

4. 樹搜尋何時贏——三場歸納的邊界條件

1. **贏的條件:blend 組合空間存在,且尚未被線性迭代充分開採。**s3e14 是唯一的勝場——線性迭代在達到 340.59891 後就依「2-3 輪增益遞減即停」的協定收手,樹搜尋額外開採出的第 6 個 blend 成員(一個單模無進步、但提供多樣性的特徵子集)是線性迭代協定下不會再花預算去試的方向。
2. **平/負的條件:線性迭代已把該指標的可用信號榨乾。**s3e5 的線性迭代已用「Optuna 目標函式直接設為 post-rounder QWK」這個全場最高槓桿配方把單模與 blend 都推到局部最優,樹搜尋的候選集(手寫佇列)沒有新方向可採,只能重新發現同一最適區(平手,差 0.00003)。s3e9 則是節點空間設計就不對(單模型),樹搜尋連「有機會贏」的入場券都沒有。
3. **離散指標需要調整 plateau 規則:**QWK-after-rounder 這類離散化指標的分數面呈階梯狀,平手頻繁、有效「改進事件」更稀疏,固定 `PLATEAU_STREAK=3` 在此比在連續指標上更快把每條 lineage 都判定為停滯(s3e5 8/8 lineage 全數 plateau,是三場中最徹底的一次,連續指標的 s3e9/s3e14 都不需要 reopen)。
4. **「blend 便宜」是指標相關,不是普遍真理:**s3e14(MAE,線性權重搜尋)的 blend 節點成本 <1s;s3e5(QWK,權重搜尋內含 Nelder-Mead 切點擬合)的 blend 節點成本 ~45-46s,與 solo 節點同量級。任何以「blend 節點近乎免費」為前提的搜尋預算規劃,必須先確認該指標的後處理成本結構。

5. 已知缺口(v1;四項已於第 6 節的 `harness_v2` 修正/實作)

- **重複子節點:**s3e5 的 blend fallback 在 parent config 不變時會對同一 parent 重複產生同一組成員(node #37/#38/#39 完全同分),在同分頻繁的離散面上浪費 plateau 額度——待修正:fallback 應檢查兄弟節點已試過的成員集合。→ v2 以 `find_duplicate_config / add_node` 內建拒絕修正(見下節「四項升級」第 3 項)。
- **blend-rounder 成本:**離散指標(QWK)下 blend 節點成本(~45-46s)追上甚至超過部分 solo 節點,「多開 blend 節點幾乎無代價」的 s3e14 經驗不能直接套用,搜尋預算分配需要按指標重新估計。→ v2 掃描證實此結論可再推廣一步:s3e11(36 萬列)顯示 blend 成本不只隨指標離散度變,也隨資料列數變(見第 8 節)。
- **無想法注入(ERA 第二支柱):**三場的每條 lineage 變異佇列都是驅動腳本裡手寫的固定清單(`CAT_QUEUE`、`XGB_QUEUE`.....),佇列用盡即退化為「換 seed」填充,搜尋期間不會依評分歷史動態產生新的候選方向。目前的「樹搜尋」本質上是「用回溯機制決定執行順序的一份預寫候選清單」,而非 ERA 描述的持續擴充候選集。→ v2 加入 `suggest_priors` (經驗庫關鍵字比對,無 LLM 呼叫)作為簡化版想法注入,見下節「四項升級」第 4 項與第 7 節——但這仍非 ERA 描述的「執行期動態生成新方向」,只是把既有經驗結構化成可查詢的候選清單,本質缺口尚未完全補上。
- **單場單樹,無跨場遷移:**三棵樹彼此獨立——沒有機制把 s3e14 學到的「blend 節點型別+加入不取代成員」直接注入下一場的初始候選集;每場都是重新手寫種子與佇列。

knowledge/experience.md (線性迭代的跨場經驗庫)目前只被拿來當作驅動腳本註解裡的「不要重試」提示,並未真正驅動候選生成。→ v2 的 `suggest_priors` 部分解決:5 場掃描中有 3 場 (s3e7/s3e19/s3e11)的經驗庫命中包含該場自己前幾輪迭代已寫入的證據,但候選集本身仍是每場手寫(見第 7 節誠實讀法)。

6. v2:四項升級與 5 場全勝掃描

四項升級(`tree_search/harness_v2.py` ,依上節缺口與 Phase C-3 舊 Stage-4 建議實作)

1. **Ensemble-default 節點空間:** `kind ( solo / blend )`升級為節點的第一級欄位(v1 藏在 `config` 裡,且要到 Phase C-2b 才追加)。OOF 快取契約正式化為 `cache_oof(cache_dir, node_id, oof, **extra) / load_oof(cache_dir, node_id)`;新增與競賽無關的 `eval_blend(cache_dir, members, metric_fn, weight_search=...)`,對已快取成員做 Dirichlet 或 grid-simplex 權重搜尋。任何後處理(四捨五入、snap-to-grid、OptimizedRounder、top-code clip、`auto_scale` 縮放……)規定必須寫在 `metric_fn` 內部,讓搜尋過程本身而非事後才套用在贏家身上——這是離散化/後處理相關指標下搜尋正確性的關鍵。
2. **Metric-aware 自適應 plateau:**新增 `tie_rate(tree)` ——量測目前所有已評分節點中「完全同分」的比例。當 `tie_rate > TIE_RATE_THRESHOLD` (0.15,取自 s3e5 QWK 的階梯面實測值 ~0.19)時,活躍 lineage 的停滯門檻從 `PLATEAU_STREAK (=3)`自動放寬為 `ADAPTIVE_PLATEAU_STREAK (=5)`,且與目前全域最佳「完全同分」的子節點視為中性(既不算改進、也不累加停滯計數),對應上節建議 2。連續指標(AUC/RMSE/SMAPE/RMSLE)下 `tie_rate` 全程接近 0,此分支從未觸發——5 場掃描全數證實,見下方結果總表。
3. **子節點去重(dedup):** `config_hash (config 的排序穩定 JSON→雜湊,演算法 sha256)+ find_duplicate_config, add_node` 預設對雜湊撞見既有節點的候選直接拒絕(回傳 `(None, dup_id)`),對應上節「重複子節點」缺口。s3e7 掃描過程中額外發現:拒絕本身不足以防止「活鎖」(fallback 連續 73 次提案同一組重複成員,全部被拒但耗用迭代預算),修正後的教訓是**提案端也要在提案前主動查詢 `find_dup`、且 `blend` 成員集合需做順序無關雜湊**——已回寫進 `run_s3e7.py / run_s3e1.py / run_s3e11.py`。
4. **經驗庫變異先驗(mutation prior):** `suggest_priors(comp_meta)` 對 `knowledge/experience.md` 的每個 `## / ###` 標題做關鍵字比對(metric/tags),逐字回傳該區塊下的證據標註 bullet(不呼叫 LLM、不做摘要),對應上節建議 3,是 ERA「想法注入」支柱的簡化版本(仍非執行期動態生成新方向——見第 7 節的誠實讀法)。

v1+v2 全 8 場結果總表

comp	harness	線性最佳	樹最佳	勝/ 平/ 負	節點	追 平 評 估 數 *	回溯	牆鐘
s3e9	v1	RMSE 12.07003	12.07459	負	20	從 未 追 平	4	125.4s
s3e14	v1	MAE 340.59891	340.52635	勝	20	9	2	593.4s
s3e5	v1	QWK 0.56769	0.56766	平	40	從 未 追 平	11(含 1 次 reopen)	696.9s
s3e3	v2	AUC 0.838140	0.841442	勝	22	7	4	34.0s
s3e7	v2	AUC 0.899893	0.900242	勝	22	8	3	405.9s
s3e1	v2	RMSE 0.557088	0.556329	勝	23(1 敗:8- member grid_simplex 超 出 5-member 上 限,ValueError 乾 淨捕捉)	9	3	190.1s
s3e19	v2	SMAPE 10.01946	9.75707	勝	22	7	2	320.9s
s3e11	v2	RMSLE 0.295648	0.295280	勝	24	10	2	513.3s

* 「追平評估數」 = 樹搜尋首次達到或優於線性最佳分數的評估序號(root = 評估 #1, 逐節點 id+1; 此為本報告統一定義, 與各場 STATUS.md 附錄行文中偶有的 off-by-one 計數習慣不必然逐字相符, 但與 docs/tree_facts.json 的 first_eval_reaching_linear_best 欄位一一對應)。以此定義複算, 五場 v2 的追平點落在評估 #7-10; 若改看「全樹最終最佳」出現的評估序號, 則落在 #12-21——這正是 s3e11 STATUS.md 「Sweep verdict」段落所述『always within 9-21 evaluations』的來源區間(原文用詞未必逐場採用同一 off-by-one 慣例, 本報告以上述統一定義複算後仍落在同一數量級)。

v2 verdict:5/5 全勝,涵蓋 4 個指標家族(AUC/RMSE/SMAPE/RMSLE)、2 種 CV 方案(KFold、TimeSeriesSplit)、3 個資料規模量級(1,677–360k 列),全部在 9–21 次評估內超越線性迭代最佳——見第 9 節結論。

7. 先驗 vs 在地洞見

`suggest_priors` 的「先驗命中率」——由經驗庫關鍵字比對出的變異(informed)相對於手寫填充變異(uninformed)的勝率——是本次掃描唯一一次系統性量化「想法注入值多少」的機會。五場的 informed vs uninformed 勝率:

comp	informed 勝率	uninformed 勝率	讀法
s3e3	14.3%	14.3%	打平——先驗與亂猜同水準
s3e7	62.5%	16.7%	先驗明顯優於 uninformed
s3e1	100%	62.5%	先驗全勝,但 uninformed 也不差
s3e19	33%	44.4%	全掃描最低——先驗反而略輸 uninformed
s3e11	100%	18.2%	先驗全勝,與 s3e1 並列全掃描最高

跨場反覆出現的發現:「先驗定下限,在地洞見定上限」。先驗(經驗庫)持續正確地「提名該試什麼方向」(seed-bagging、加入不取代、移除近零權重成員、regularize-over-capacity 等在 4/5 場都轉移成功),其價值主要是**避免浪費算力在已知死路上**(如 s3e3 沒有再去重調 CatBoost、s3e7 沒有再走不轉移的 regularize-over-capacity)。但每一場**真正拉開分數差距的最大單一槓桿,始終來自該場自己的 EDA/comp-local 洞見,而非經驗庫**——四個具體例子:

- **s3e3:tenure-prune**——砍掉已被工程特徵取代的 5 個原始 tenure 欄位 (+ income_per_year_worked),0.838903→0.841442(全樹最終最佳),此為該場 STATUS.md 自己「Next ideas」清單裡的未驗證假設,不是經驗庫命中。
- **s3e1:top-code clip**——在 blend 權重搜尋內部(而非只在最終 test 提交時)對 OOF 套用 top-code 感知的 clip,0.557054→0.556931,單一槓桿源自該場 EDA 發現的「4.92% 目標值頂在 5.00001」現象。
- **s3e19:auto_scale**——時序 OOF 系統性偏低(驗證窗永遠晚於訓練窗、序列持續成長)的一個全域 ×1.02 純量修正,單步就把 8-way blend 從 9.943698 拉到 **9.775774**(全跑最大單一增益),經驗庫裡沒有時序特有的這條規則。
- **s3e11:depth-boundary push**——Optuna 自己把 CatBoost depth 搜到上界 10 就停了;人工把 depth 推到 12,solo 0.295779→0.295461,勝過整個 phase-1 blend,延伸出「當 Optuna 最優解卡在搜尋空間邊界上,邊界本身就是下一個變異」的新推論。

8. 誠實但書

v1 三場的既有但書(第 3 節逐場分析已個別交代,此處彙總):s3e9 樹搜尋結構性到不了 blend 空間(節點空間設計問題,非搜尋機制問題);s3e5 的「平手」量級(差 0.00003)小到單一樣本切點歸屬就能翻轉,不宜過度解讀為機制等價;三棵樹之間無知識遷移,每場種子與變異佇列皆手寫。

v2 五場新增的但書:

1. **全部 8 場、v1+v2,沒有一次提交 Kaggle**——本次週末批次執行「不碰任何 token」是鐵則(見 `.superpowers/weekend-plan.md`),所有「樹最佳勝過線性最佳」的結論都只在 **OOF(out-of-fold)** 分數上成立,尚未經 Public/Private Leaderboard 驗證。OOF 勝出不保證 LB 勝出,尤其在權重搜尋/後處理參數本身就是對同一份 OOF 擬合出來的情況下(見下一條)。
2. **s3e19 的 fold-5 double-dip + OOF-fitted 但書,原樣繼承並加重:**線性迭代原本的 10.01946 已帶有「fold 5 同時是 Optuna 調參目標、又是 5 折 OOF 的其中一折」的雙重使用樂觀偏誤;樹搜尋的 9.75707 在此之上再疊加兩層 OOF 擬合——`auto_scale` ($\times 1.02$ 純量)與種子選擇本身都是直接對 114,000 列 OOF 擬合出來的 1 至少數參數。單獨看每一層都是低過擬合風險的簡單擬合,但誠實的讀法是「2022 年實際 SMAPE 應顯著低於 10.02,但不應直接讀成 9.76」。
3. **v2 掃描期間曾出現一次搜尋層級的活鎖(s3e7):**blend fallback 在成員佇列耗盡後連續 73 次提出同一組重複成員,雖然每次都被 dedup 正確拒絕、零算力浪費,但迭代預算被無謂消耗至安全上限提前結束該輪搜尋(15/22 節點)——已修正(提案端預先查詢 + 成員集合順序無關雜湊),但這代表「拒絕型 dedup」單獨並不足夠,純屬工程層面的教訓而非分數層面的但書。
4. **經驗庫的「先驗」有時只是該場自己先前迭代的回饋:**s3e19/s3e11 兩場的 `suggest_priors` 命中結果裡,相當比例的證據列其實正是該場自己 Phase B 迭代寫入 `knowledge/experience.md` 的結論——嚴格說不算「跨場遷移」,是該場自己餵自己。真正的跨場遷移證據要看 s3e3/s3e7/s3e1 命中的、來自其他場次的 bullet。
5. **想法注入仍未真正實現:**上節「四項升級」第 4 項的 `suggest_priors` 只是關鍵字比對既有經驗庫的靜態清單,搜尋期間不會依評分歷史動態生成新方向——第 7 節「先驗定下限、在地洞見定上限」的發現,某種意義上正是這個缺口尚未補上的直接證據:經驗庫給得出「該試什麼類別」,但給不出本次掃描裡任何一場的最大單一增益。

9. 結論與 Stage 4 建議(v1+v2)

v2 verdict

5/5 全勝:s3e3(AUC,小樣本)、s3e7(AUC)、s3e1(RMSE,地理特徵)、s3e19(SMAPE,TimeSeriesSplit)、s3e11(RMSLE,36 萬列)。`harness_v2` (ensemble-default 節點空間 + OOF 快取 + 經驗庫先驗 + 自適應 plateau + 子節點去重)在本次測試的每一個指標家族、每一種 CV 方案(KFold 與 TimeSeriesSplit)、每一個資料規模量級上都勝過線性迭代最佳,且都在 9-21 次評估內達成(見上節結果總表附註對此區間的精確定義)。相較於 Phase C-3 的 1 勝 1 平 1 負,v2 的四項升

級(尤其是 ensemble-default 節點空間——不再需要先讓單模型 plateau 才加 blend)是勝率躍升的主要原因。

v3 候選規則(本次掃描過程中發現、尚未實作的下一版迭代方向)

1. **plateau 後的 solo 突破應重新開啟已 plateau 的 blend lineage**:s3e11 的全樹最終最佳來自 phase-1 BLEND lineage 早已 plateau 之後才出現的 depth-12 CatBoost solo;現行機制不會自動把新 solo 帶回一個已標記 plateau 的 blend lineage 重新混合,本次是靠人工開了「phase 2」重新搜尋才拿到 0.295280——這應該是 harness 的標準行為,而非人工介入。
2. **Optuna 邊界檢查應成為標準變異節點**:s3e11 顯示「當 Optuna 的最優解卡在搜尋空間邊界上(depth 搜 4-10、選中 10),邊界本身就是下一個該試的方向」——建議 harness 對每個 Optuna 調校過的 solo 節點,自動生成一個「往搜索邊界外推一步」的變異候選,而非依賴人工發現。
3. **TimeSeriesSplit 下應放大 seed-bagging 的優先權**:s3e19 顯示同一組 Optuna 調校超參數,光是換 random seed 就能讓 solo 分數從 root(seed 42)的 **10.148325** 跳到 SEEDBAG_TUNED(seed 2024)的 **9.974778**——seed 間擴散遠大於 KFold 場次,因為 fold 不可互換放大了模型變異——seed-bagging 在 TS 場次的預期報酬結構性更高,mutation 佇列/ suggest_priors 應該依 CV 方案調整 seed-bagging 的建議優先序,而非套用與 KFold 相同的權重。
4. **blend 節點成本需同時感知指標與資料規模**:第 5 節缺口延續、本次掃描再擴充一步的證據——s3e11(36 萬列)的 blend 節點成本(17-39s)已不再是「近乎免費」,而是隨列數線性增長;搜尋預算規劃需要同時考慮「指標是否離散化」(第 5 節原有發現)與「資料列數」(本次新增發現)兩個維度,而非只看前者。

10. 重現指令(v1+v2)

```
cd /home/tjyen/ai_agents/kaggle

# --- v1(harness.py,Phase C-2a/b/c) ---
# s3e9 - 單模型節點空間
uv run python3 tree_search/run_s3e9.py

# s3e14 - solo + blend 節點空間
uv run python3 tree_search/run_s3e14.py

# s3e5 - 離散化指標(QWK-after-rounder)泛化測試
uv run python3 tree_search/run_s3e5.py

# --- v2(harness_v2.py,Phase D-2..D-6,依資料規模由小到大排序) ---
```

```
uv run python3 tree_search/run_s3e3.py      # AUC,小樣本(1,677 列)
uv run python3 tree_search/run_s3e7.py      # AUC,42k 列
uv run python3 tree_search/run_s3e1.py      # RMSE,地理特徵,37k 列
uv run python3 tree_search/run_s3e19.py     # SMAPE,TimeSeriesSplit
uv run python3 tree_search/run_s3e11.py     # RMSLE,36 萬列(全掃描最大規模)
```

八支腳本皆可中斷/續跑(每個節點寫入後立即以 temp-file + os.replace 原子寫入 experiments_tree.json)。若要從零開始重跑,先刪除對應的 competitions/playground-series-<comp>/experiments_tree.json 與 tree_search/cache_<comp>/ (皆為可重新產生、已 gitignore 的暫存)。

事實抽取與驗證:

```
uv run python3 docs/scripts/build_tree_facts.py
uv run python3 .claude/skills/kaggle-report/assets/verify_report.py docs/tree_search
bash .claude/skills/kaggle-report/assets/md2pdf.sh docs/tree_search_prototype.md
```